

CoachFit Cut Assistant — Trade-off Analysis

LIU ZEYUAN · PE6201 Section B · End-of-Course Project · Code: github.com/Lizzy-808/coachfit-cut-assistant · Live: coachfit-cut-assistant.streamlit.app

1. The decision

A gym coach needs to know, per client, whether daily intake is a sensible fat-loss deficit. In my system **deterministic rules make every decision and a rented language model only explains it**: rules compute BMR, TDEE, the deficit label and safety flags; gpt-4o-mini writes a short note for the coach; two guards I wrote check it: a number the rules did not produce sends the note back to a fixed template, and a flag it leaves out is appended by code. Out of scope: food logs, macronutrient targets, and protein (section 4 explains why).

2. Alternatives compared

Option	Correct verdict	US\$ per successful check	Main failure mode
Coach by hand	—	4.89	5–10 minutes; coaches round differently
LLM classifies directly (formula in prompt)	37–40%	~3.00†	Silent arithmetic errors
Rules + fixed template	100%*	1.08	Stiff wording
Rules + LLM note + guards (chosen)	100%*	1.08–1.75‡	Fluent note that misstates a relation

*Against labels computed independently in Excel on 787 valid NHANES rows — this checks my arithmetic, not the model; the model is measured in section 4. Majority baseline 25%. †Assumes errors are caught; they are silent, so the true cost is higher. ‡At the confirmatory 25/25 and at its 95% lower bound, 86%.

Given the formula, the LLM got one case in three right, and one in five within 100 kcal of a boundary — where a coach most needs help. It is an arithmetic task dressed as a language task, so the model belongs after the decision. The honest counterpoint is row three: the template is correct and free, so the LLM can only add readability — section 3 prices how much.

3. Is it worth building? (Class 5)

Cost per successful check = variable cost + (1 – success rate) × cost of a failure. A failure is a note the coach must redo by hand: 7.5 minutes at S\$50 an hour (one PT session at my gym), US\$4.89 at 0.7823 USD/SGD on 28 September. Tokens cost US\$0.00014 per note — irrelevant. What dominates is the 22% of real checks the rules send to the coach (mostly intake below the safety floor) and the note's error rate. Unguarded, the model's notes passed 40% of the time, so each would have to save a coach **270 seconds** to beat the template; guarded, at the 86% lower bound, about **62 seconds**. That is my kill condition, set before any coach trial: if notes do not read that much faster than the template, drop the LLM.

What it buys. Scale only — an efficiency case, defended as one. It fails the free-label test: weigh-ins would show whether an estimate was right, but they are self-reported and never flow back automatically. Scope is weak: no second team needs this pipeline yet.

Data readiness. NHANES passes accessibility and standardisation, but real gym data fails *quality* and *consistency*: one-day recalls under-report so often that 18.5% of NHANES adults fall below the 1,200/1,500 kcal floor, and client logs vary by app. The operating model — who records intake, weekly, with PDPA consent — is the bottleneck, not the model.

4. What the evaluation changed

Exploratory runs shaped the design. Run 1's guard rejected 42 of 226 notes for "invented numbers" — all false positives: my regex read "-90 kcal" as 90. The judge, gpt-4o-mini itself, failed correct advice; its replacement, deepseek-chat, agreed with my 20 hand labels only 75% of the time (FAIL precision 60%, recall 86%), so judge scores are not my metric. Hand grading showed the rules themselves confused the model: flagging "below BMR" beside an *appropriate* label made it say "eat more"; rewording the flag cut that from 4/10 to 0/10. On the instructor's suggested metric — does the note state every flag the rules raised, and invent none? — v2 scored 10/20, all omissions. Six were "protein not logged": my NHANES extract lost its protein column in A1, so I removed protein from scope rather than ship an unvalidated rule. The rest shaped v3's one change: code appends any flag the note left out. These runs mixed scopes, cases and an unblinded grader, so they cannot compare v2 with v3.

Confirmatory test. I committed the protocol, pass criteria and analysis script before generating any data. On 25 unused NHANES cases the model was called once each; version A is its note, version B the same note after the guard, so they differ only by the guard. I graded all 50, shuffled and unlabelled, flag by flag.

Pre-registered measure	Result	Pass line
Guarded notes stating every flag, inventing none	25/25 (95% CI 86–100%)	≥ 24/25 ✓
Omitted flags the detector caught	16/16	≥ 95% ✓
Flags invented or contradictions added by the guard	0	0 ✓
Unguarded notes passing	10/25 (21–61%)	—

No note got worse or invented a flag; the model omitted "below BMR" (10) and the logging-error warning (6) — flags it judges unimportant, exactly the choice the guard takes away. Caveats: I am both author and grader. I graded twice — a quick pass, then a slower one declared primary before I read it (logged as a deviation); they agreed on all 172 marks and matched the detector on every note. The appended sentences are recognisable, so blinding is partial.

Abstention became a range. Flagging cases within 50 kcal of a boundary caught only 28% of those a $\pm 10\%$ logging error would flip; 48% of real cases are that sensitive, and catching 76% would have sent 38% of clients to review. The tool shows every label reachable within $\pm 10\%$ and forces review only for safety cases — all 27 routed correctly.

5. Build vs buy

Own the decision logic (24 unit tests), guards and evaluation. Rent the model (gpt-4o-mini via OpenRouter, swappable in one line), the judge and Streamlit. A competitor could rent all of that by next Tuesday; what they could not quickly copy — consented client data inside a gym's weekly workflow — this project does not yet have. Low-code was not tried: unit-tested arithmetic does not fit a no-code builder. All evaluation cost US\$0.31.

6. Limitations and when the answer changes

The four labels are finer than the data: a 10% logging error is as wide as the "appropriate" band. Every hand grade is mine; a coach-graded set is the missing check. No real coach has used the tool, so the time saving — the whole business case — is unmeasured. Responsible use: healthy adults only, under-18s refused, no identifiers sent to the model, human review for safety cases (IMDA framework), guards against misinformation (OWASP LLM09). If coaches entered free-text food logs, the model would move into the decision path and need its own accuracy evaluation.

7. What changes, and what next

Against today. A coach now skips the calculation or spends 5–10 minutes per client; a chatbot answers confidently and is right a third of the time. With the tool, every client is checked the same way in seconds, logging uncertainty is visible, unsafe intake always reaches a person, and the note cannot drop a flag. Whether it saves the coach time is not yet shown.

Next: calibrate TDEE from weigh-ins. The weakest number is TDEE — a formula times a self-reported activity factor. If the gym's scale logged weekly weights, the trend against logged intake would reveal each client's real expenditure within a month, assuming their logging bias is stable. Weigh-ins recorded by the gym, not reported by the client, are a label the world supplies for free — the first step from an efficiency tool towards one that learns. The test: does calibrated TDEE predict next week's weight better than the formula?

Tools: Claude (Anthropic) helped write code and draft this text; ChatGPT translated evaluation material for reading. All figures come from `eval/results/` and `eval/confirmatory/` (protocol and deviations logged there).